

CSC 281

Collins, Trevon Submission At End

Objective

In this assignment, you will implement the Fibonacci sequence in two different ways:

1. Recursive implementation
2. Iterative implementation using a loop

The goal is to compare performance and understand when to use each approach. All implementations must be written in Java.

Reminder: Fibonacci Definition

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

Base cases: $\text{fib}(0)=0$, $\text{fib}(1)=1$

Part 1: Implement Fibonacci in Two Ways

- A. Recursive version
- B. Iterative version

Note:

- You must write both the recursive and iterative versions in Java.
- Use standard Java libraries only.
- Do not use built-in functions or external libraries that compute Fibonacci for you.

Part 2: Compute Fibonacci Values

Test at least: $n = 5, 10, 15, 20, 25, 30, 35, 40$

Also attempt $\text{fib}(100)$ using both methods.

Part 3: Measure Running Time

Record running time for each method and value of n .

Include a table with:

- n
- Recursive result
- Recursive time
- Iterative result
- Iterative time

Part 4: Create a Line Chart

Create a chart with:

- x-axis: n
- y-axis: running time
- one line for recursion
- one line for iteration

Part 5: Answer Questions

- Did both implementations produce the same results?
- Which implementation was faster? Why?
- How did recursive time change as n increased?
- How did iterative time change as n increased?
- Could you compute fib(100) recursively?
- Which implementation is better for large n and why?
- What repeated work occurs?

Submission Requirements

1. Source code
2. Report (with table, chart, answers)
3. Screenshot of program output

Java Source Code

```
public class Homework02
{
    static long start, end;
    static double fibElapsed, iterElapsed;
    public static void main(String[] args) {
        int[] fibArray = {5, 10, 15, 20, 25, 30, 35, 40, 100};
        printTable(fibArray);
    }

    public static void printTable(int[] _fibArray) {
        System.out.println("|Fibonacci Number (n)|");
        for (int i : _fibArray) {
            System.out.printf("%d|%d|%.1f|%d|%.1f|%n",
i,fibRecursiveTrack(i), fibElapsed, fibIterative(i),
iterElapsed);
            System.out.println("-----");
        }
        System.out.println("=====");
    }

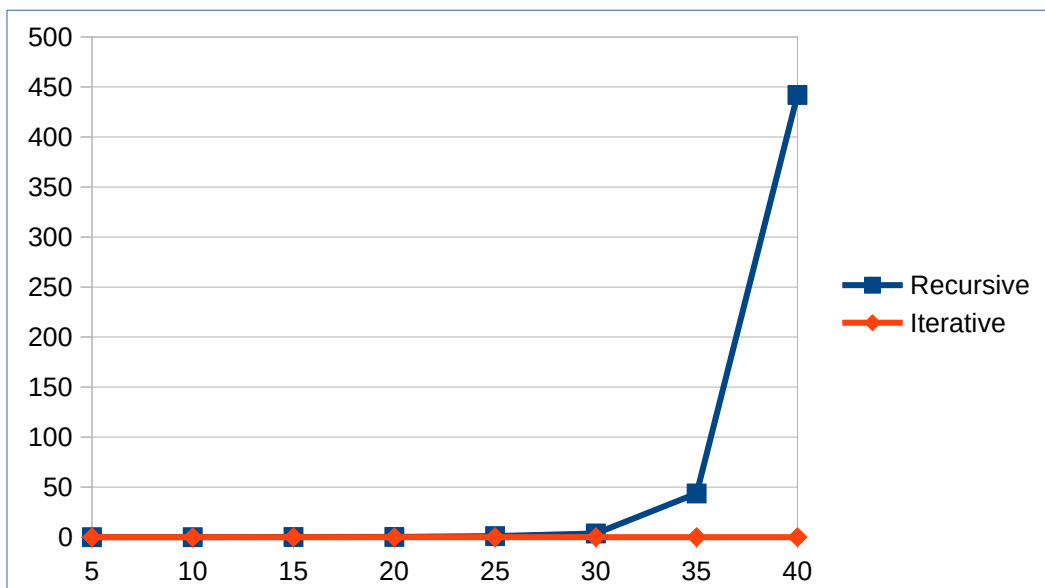
    public static int fibRecursiveTrack(int n){
        start = System.nanoTime();
        int fibResult = fibRecursive(n);
        end = System.nanoTime() - start;
        fibElapsed = end / 1_000_000.0;
        return fibResult;
    }
}
```

```
public static int fibRecursive(int n) {  
    if (n == 0){  
        return 0;  
    }  
    if (n == 1){  
        return 1;  
    }  
    return fibRecursive(n-1) + fibRecursive(n-2);  
}
```

```
public static int fibIterative(int n){  
    start = System.nanoTime();  
    int[] iterArray = new int[n+1];  
    for (int i = 0; i <= n; i++){  
        if (i == 0){  
            iterArray[i] = 0;  
        } else if (i == 1){  
            iterArray[i] = 1;  
        } else {  
            iterArray[i] = iterArray[i-1] + iterArray[i-2];  
        }  
    }  
    end = System.nanoTime() - start;  
    iterElapsed = end / 1_000_000;  
    return iterArray[n];  
}
```

Graph

	Recursive Result	Recursive Time Elapsed	Iterative Result	Iterative Time Elapsed
5	5	0ms	5	0ms
10	55	0ms	55	0ms
15	610	0.1ms	610	0ms
20	6765	0.2ms	6765	0ms
25	75025	1.0ms	75025	0ms
30	832040	3.7ms	832040	0ms
35	9227465	43.6ms	9227465	0ms
40	102334155	442.0ms	102334155	0ms
100	Undetermined	~10 yrs*	Too large	0ms



- Did both implementations produce the same results? They both use the same values that would depend on the results of previous entries.
- Which implementation was faster? Why? The iterative implementation was faster because the for loop can use stored information unlike the recursive that always calculates in run time.
- How did recursive time change as n increased? It began to increase exponentially with every
- How did iterative time change as n increased? It remained constant
- Could you compute fib(100) recursively? No.
- Which implementation is better for large n and why? Iterative is better for large n because it can utilize stored memory from the device.
- What repeated work occurs? The mathematics of the previous entries.